

4-Week DSA Homework Plan

WEEK 1 — Arrays + Basic Strings

Day 1: Array Basics

1. Two Sum
<https://leetcode.com/problems/two-sum/>
2. Remove Duplicates from Sorted Array
<https://leetcode.com/problems/remove-duplicates-from-sorted-array/>
3. Best Time to Buy and Sell Stock
<https://leetcode.com/problems/best-time-to-buy-and-sell-stock/>

Day 2: Prefix Sum / Sliding Window

1. Maximum Subarray
<https://leetcode.com/problems/maximum-subarray/>
2. Contains Duplicate
<https://leetcode.com/problems/contains-duplicate/>
3. Maximum Average Subarray I
<https://leetcode.com/problems/maximum-average-subarray-i/>

Day 3: Two Pointer

1. Move Zeroes
<https://leetcode.com/problems/move-zeroes/>
2. Squares of a Sorted Array
<https://leetcode.com/problems/squares-of-a-sorted-array/>
3. Container With Most Water
<https://leetcode.com/problems/container-with-most-water/>

Day 4: Matrix Problems

1. Matrix Diagonal Sum
<https://leetcode.com/problems/matrix-diagonal-sum/>
2. Reshape the Matrix
<https://leetcode.com/problems/reshape-the-matrix/>
3. Spiral Matrix
<https://leetcode.com/problems/spiral-matrix/>

Day 5: String Basics

1. Valid Palindrome
<https://leetcode.com/problems/valid-palindrome/>
2. Reverse String
<https://leetcode.com/problems/reverse-string/>
3. Longest Common Prefix
<https://leetcode.com/problems/longest-common-prefix/>

OOPS Problem of the Week: -

TOPIC IS — INTERFACE

A library needs to develop an online application for two types of users/roles, Adults and children. Both of these users should be able to register an account.

Any user who is less than 12 years of age will be registered as a child and they can borrow a “Kids” category book for 10 days, whereas an adult can borrow “Fiction” category books which need to be returned within 7 days.

Note: In future, more users/roles might be added to the library where similar rules will be enforced.

Develop Interfaces and classes for the categories mentioned above.

1. Create an interface LibraryUser with the following methods declared,

Method Name

registerAccount

requestBook

2. Create 2 classes “KidUsers” and “AdultUser” which implements the LibraryUser interface.

3. Both the classes should have two instance variables as specified below.

Instance variables Data type

age int

bookType String

4. The methods in the KidUser class should perform the following logic.

registerAccount function:

if age < 12, a message displaying “You have successfully registered under a Kids Account” should be displayed in the console.

If(age>12), a message displaying, “Sorry, Age must be less than 12 to register as a kid” should be displayed in the console.

requestBook function:

if bookType is "Kids", a message displaying "Book Issued successfully, please return the book within 10 days" should be displayed in the console.

Else, a message displaying, "Oops, you are allowed to take only kids books" should be displayed in the console.

5. The methods in the AdultUser class should perform the following logic.

registerAccount function:

if age > 12, a message displaying "You have successfully registered under an Adult Account" should be displayed in the console.

If age < 12, a message displaying, "Sorry, Age must be greater than 12 to register as an adult" should be displayed in the console.

requestBook function:

if bookType is "Fiction", a message displaying "Book Issued successfully, please return the book within 7 days" should be displayed in the console.

Else, a message displaying, "Oops, you are allowed to take only adult Fiction books" should be displayed in the console.

6. Create a class "LibraryInterfaceDemo.java" with a main method which performs the below functions,

Test case #1:

Create an instance of KidUser class.

Set the age as specified in the below table and invoke the registerAccount method of the KidUser object

Age

10

18

Set the book Type as specified in the below table and invoke the requestBook method of the KidUser object,

BookType

"Kids"

"Fiction"

Test case #2:

Create an instance of AdultUser class.

Set the age as specified in the below table and invoke the registerAccount method of the AdultUser object

Age

5

23

Set the book Type as specified in the below table and invoke the requestBook method of the AdultUser object

BookType

“Kids”

“Fiction”

WEEK 2 — Strings (Advanced)

Day 1: Hashing in Strings

1. Valid Anagram
<https://leetcode.com/problems/valid-anagram/>
2. First Unique Character in a String
<https://leetcode.com/problems/first-unique-character-in-a-string/>
3. Ransom Note
<https://leetcode.com/problems/ransom-note/>

Day 2: Sliding Window on Strings

1. Longest Substring Without Repeating Characters
<https://leetcode.com/problems/longest-substring-without-repeating-characters/>
2. Permutation in String
<https://leetcode.com/problems/permutation-in-string/>
3. Find All Anagrams in a String
<https://leetcode.com/problems/find-all-anagrams-in-a-string/>

Day 3: Pattern Matching

1. Implement strStr()
<https://leetcode.com/problems/find-the-index-of-the-first-occurrence-in-a-string/>
2. Is Subsequence
<https://leetcode.com/problems/is-subsequence/>

3. Repeated Substring Pattern
<https://leetcode.com/problems/repeated-substring-pattern/>

Day 4: Recursion + Strings

1. Reverse Words in a String III
<https://leetcode.com/problems/reverse-words-in-a-string-iii/>
2. Decode String
<https://leetcode.com/problems/decode-string/>
3. Generate Parentheses
<https://leetcode.com/problems/generate-parentheses/>

Day 5: Mixed String Problems

1. Group Anagrams
<https://leetcode.com/problems/group-anagrams/>
2. String Compression
<https://leetcode.com/problems/string-compression/>
3. Longest Palindromic Substring
<https://leetcode.com/problems/longest-palindromic-substring/>

OOPS Problem of the Week: -

Write an interface called Playable, with a method

`void play();`

Let this interface be placed in a package called music.

Write a class called Veena which implements Playable interface. Let this class be placed in a package music.string

Write a class called Saxophone which implements Playable interface. Let this class be placed in a package music.wind

Write another class Test in a package called live. Then,

- a. Create an instance of Veena and call play() method
- b. Create an instance of Saxophone and call play() method
- c. Place the above instances in a variable of type Playable and then call play()

WEEK 3 — Linked List + Stack + Queue

Part A — Linked List

Day 1

1. Linked List Cycle
<https://leetcode.com/problems/linked-list-cycle/>
2. Reverse Linked List
<https://leetcode.com/problems/reverse-linked-list/>
3. Middle of the Linked List
<https://leetcode.com/problems/middle-of-the-linked-list/>

Day 2

1. Merge Two Sorted Lists
<https://leetcode.com/problems/merge-two-sorted-lists/>
2. Remove Nth Node From End of List
<https://leetcode.com/problems/remove-nth-node-from-end-of-list/>
3. Palindrome Linked List
<https://leetcode.com/problems/palindrome-linked-list/>

Part B — Stack

Day 3

1. Valid Parentheses
<https://leetcode.com/problems/valid-parentheses/>
2. Min Stack
<https://leetcode.com/problems/min-stack/>
3. Next Greater Element I
<https://leetcode.com/problems/next-greater-element-i/>

Day 4

1. Daily Temperatures
<https://leetcode.com/problems/daily-temperatures/>
2. Evaluate Reverse Polish Notation
<https://leetcode.com/problems/evaluate-reverse-polish-notation/>
3. Largest Rectangle in Histogram
<https://leetcode.com/problems/largest-rectangle-in-histogram/>

Part C — Queue / Deque

Day 5

1. Implement Queue using Stacks
<https://leetcode.com/problems/implement-queue-using-stacks/>
2. Number of Recent Calls
<https://leetcode.com/problems/number-of-recent-calls/>
3. Sliding Window Maximum
<https://leetcode.com/problems/sliding-window-maximum/>

OOPS Problem of the Week: -

Create an abstract class `Compartment` to represent a rail coach. Provide an abstract function `notice` in this class.

```
public abstract String notice();
```

Derive `FirstClass`, `Ladies`, `General`, `Luggage` classes from the `compartment` class. Override the `notice` function in each of them to print notice message that is suitable to the specific type of compartment.

Create a class `TestCompartment`. Write main function to do the following:

Declare an array of `Compartment` of size 10.

Create a compartment of a type as decided by a randomly generated integer in the range 1 to 4.

Check the polymorphic behavior of the `notice` method.

[i.e based on the random number generated, the first compartment can be `Luggage`, the second one could be `Ladies` and so on..]

WEEK 4 — Trees

Day 1: Binary Tree Basics

1. Maximum Depth of Binary Tree
<https://leetcode.com/problems/maximum-depth-of-binary-tree/>
2. Invert Binary Tree
<https://leetcode.com/problems/invert-binary-tree/>
3. Same Tree
<https://leetcode.com/problems/same-tree/>

Day 2: Traversals

1. Binary Tree Inorder Traversal
<https://leetcode.com/problems/binary-tree-inorder-traversal/>
2. Binary Tree Level Order Traversal
<https://leetcode.com/problems/binary-tree-level-order-traversal/>

3. Binary Tree Zigzag Level Order Traversal
<https://leetcode.com/problems/binary-tree-zigzag-level-order-traversal/>

Day 3: Binary Search Tree

1. Search in a Binary Search Tree
<https://leetcode.com/problems/search-in-a-binary-search-tree/>
2. Validate Binary Search Tree
<https://leetcode.com/problems/validate-binary-search-tree/>
3. Lowest Common Ancestor of a BST
<https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-search-tree/>

Day 4: Path Based Problems

1. Path Sum
<https://leetcode.com/problems/path-sum/>
2. Diameter of Binary Tree
<https://leetcode.com/problems/diameter-of-binary-tree/>
3. Binary Tree Maximum Path Sum
<https://leetcode.com/problems/binary-tree-maximum-path-sum/>

Day 5: Advanced Trees

1. Symmetric Tree
<https://leetcode.com/problems/symmetric-tree/>
2. Construct Binary Tree from Preorder and Inorder Traversal
<https://leetcode.com/problems/construct-binary-tree-from-preorder-and-inorder-traversal/>
3. Serialize and Deserialize Binary Tree
<https://leetcode.com/problems/serialize-and-deserialize-binary-tree/>

OOPS Problem of the Week: -

1. Write a program to create interface named test. In this interface the member function is square. Implement this interface in arithmetic class. Create one new class called ToTestInt in this class use the object of arithmetic class.
2. Create an outer class with a function display, again create another class inside the outer class named inner with a function called display and call the two functions in the main class.
3. Create class point with following instance variable and methods.

Instance variable: private int x,y

Constructors : public Point(), Point(int x, int y)

Methods : public void setX(int x), setY(int y), setXY(int x, int y)

4. Create class box and box3d. box3d is extended class of box. The above two classes going to pull fill following requirement

Include constructor.

set value of length, breadth, height Find out area and volume.

Note: Base class and sub classes have respective methods and instance variables.